Perfect 🚀 Let's continue with **Module 21: Triggers**.

📘 SQL CLASS NOTE – MODULE 21: TRIGGERS

1. Introduction

A **Trigger** in SQL is a **special stored program** that automatically executes in response to certain events on a table.

👉 Events can be:

- `INSERT`
- `UPDATE`
- `DELETE`

👉 Triggers are useful for:

- Enforcing **business rules**
- Maintaining **audit logs**
- Ensuring **data integrity**

2. Syntax

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
FOR EACH ROW
BEGIN
   -- SQL statements
END;
```

- **BEFORE**: Trigger runs **before** the operation.
- **AFTER**: Trigger runs **after** the operation.
- **OLD**: Refers to the row before the change (UPDATE/DELETE).

- **NEW**: Refers to the row after the change (INSERT/UPDATE).

3. Practical Examples with `SchoolDB`

Example 1: Audit log when a student is inserted

```
CREATE TABLE StudentAudit (
   AuditID INT AUTO_INCREMENT PRIMARY KEY,
   ActionType VARCHAR(20),
   StudentID INT,
   ActionDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TRIGGER after_student_insert
AFTER INSERT ON Students
FOR EACH ROW
BEGIN
   INSERT INTO StudentAudit (ActionType, StudentID)
   VALUES ('INSERT', NEW.StudentID);
END;
```

Example 2: Prevent deleting students who are still enrolled

```
CREATE TRIGGER prevent_student_delete
BEFORE DELETE ON Students
FOR EACH ROW
BEGIN
   IF EXISTS (SELECT 1 FROM Enrollments WHERE StudentID = OLD.StudentID) THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot delete student who is enrolled in a course';
   END IF;
END;
```

Example 3: Automatically set default grade when inserting a student

```
CREATE TRIGGER set_default_grade
BEFORE INSERT ON Students
FOR EACH ROW
BEGIN
   IF NEW.GradeLevel IS NULL THEN
      SET NEW.GradeLevel = 'Grade 9';
   END IF;
END;
```

Example 4: Update audit log when a student's age is updated

```
CREATE TRIGGER after_student_update
AFTER UPDATE ON Students
FOR EACH ROW
BEGIN
   INSERT INTO StudentAudit (ActionType, StudentID)
   VALUES ('UPDATE', NEW.StudentID);
END;
```

Example 5: Log deleted students

```
CREATE TRIGGER after_student_delete
AFTER DELETE ON Students
FOR EACH ROW
BEGIN
   INSERT INTO StudentAudit (ActionType, StudentID)
   VALUES ('DELETE', OLD.StudentID);
END;
```

4. Why Use Triggers?

- ✅ Enforce **automatic rules**.
- ✅ Maintain **data accuracy**.

- ✅ Keep **audit trails**.
- ✅ Prevent invalid operations.

5. Real-Life Use Cases

- **Schools**: Auto-log student insertions, updates, or deletions.
- **Banks**: Track every deposit/withdrawal in a log.
- **E-commerce**: Update inventory automatically after purchase.
- **Hospitals**: Record patient discharge automatically when deleted.

6. Classwork (Practice) ✅

1. Create a trigger that logs student inserts into `StudentAudit`.
2. Create a trigger that prevents deleting a student if they are enrolled.
3. Create a trigger that sets `GradeLevel = 'Grade 9'` if no grade is given.
4. Create a trigger that logs updates on the `Students` table.
5. Create a trigger that logs deletions on the `Students` table.

7. Challenge Question 💡

👉 Create a trigger called `update_course_count` that automatically updates the total number of students enrolled in a course whenever a new enrollment is added.

✅ That's the **expanded Module 21: Triggers**.

👉Perfect 👍 Let's solve the **Module 21 Classwork + Challenge** step by step with SQL trigger definitions.

✅ Solutions to Module 21: `TRIGGERS`

1. Create a trigger that logs student inserts into `StudentAudit`.

```
CREATE TRIGGER trg_after_student_insert
AFTER INSERT ON Students
FOR EACH ROW
BEGIN
   INSERT INTO StudentAudit (ActionType, StudentID)
   VALUES ('INSERT', NEW.StudentID);
END;
```

2. Create a trigger that prevents deleting a student if they are enrolled.

```
CREATE TRIGGER trg_prevent_student_delete
BEFORE DELETE ON Students
FOR EACH ROW
BEGIN
   IF EXISTS (SELECT 1 FROM Enrollments WHERE StudentID = OLD.StudentID) THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot delete student who is enrolled in a course';
   END IF;
END;
```

3. Create a trigger that sets `GradeLevel = 'Grade 9'` if no grade is given.

```
CREATE TRIGGER trg_set_default_grade
BEFORE INSERT ON Students
FOR EACH ROW
BEGIN
   IF NEW.GradeLevel IS NULL THEN
      SET NEW.GradeLevel = 'Grade 9';
   END IF;
END;
```

4. Create a trigger that logs updates on the `Students` table.

```
CREATE TRIGGER trg_after_student_update
AFTER UPDATE ON Students
FOR EACH ROW
BEGIN
   INSERT INTO StudentAudit (ActionType, StudentID)
   VALUES ('UPDATE', NEW.StudentID);
END;
```

5. Create a trigger that logs deletions on the `Students` table.

```
CREATE TRIGGER trg_after_student_delete
AFTER DELETE ON Students
FOR EACH ROW
BEGIN
   INSERT INTO StudentAudit (ActionType, StudentID)
   VALUES ('DELETE', OLD.StudentID);
END;
```

💡 Challenge Question

👉 Create a trigger called `update_course_count` that updates the total number of students in a course whenever a new enrollment is added.

Step 1: Add a column `TotalEnrollments` to `Courses` (if not already there).

```
ALTER TABLE Courses ADD COLUMN TotalEnrollments INT DEFAULT 0;
```

Step 2: Create the trigger.

```
CREATE TRIGGER trg_update_course_count
AFTER INSERT ON Enrollments
FOR EACH ROW
BEGIN
   UPDATE Courses
   SET TotalEnrollments = TotalEnrollments + 1
   WHERE CourseID = NEW.CourseID;
END;
```

📌 Now, whenever a new enrollment is added, the related course's student count is automatically updated.

✅ That completes **Module 21: Triggers**.